PGBD - PROJET DE DÉVELOPPEMENT SGBD

Architectures applicatives:
Principes de base et bonnes pratiques (partie 2)

Frédéric Parion
f.parion@esa-namur.be

1

ARCHITECTURES APPLICATIVES

Organisation en couches

2

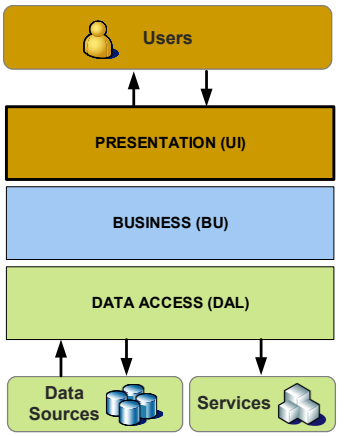
Organisation en couches

La couche présentation (1/4)

La **couche présentation** constitue l'interface utilisateur de l'applicat° et gère les interactions avec les utilisateurs.

Elle est constituée de **UI Components** (composants d'interface) qui peuvent être:

- Composants maui, wpf, winforms,...
- Des views/controllers mvc, webforms...
- Une application console
- ...



3

3

Organisation en couches

La couche présentation (2/4)

Responsabilités de la couche présentation :

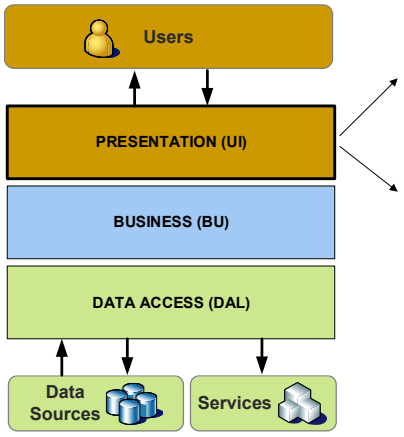
- Afficher les données à l'utilisateur
- Acquérir et valider les données à partir des entrées utilisateur
- Interpréter les évènements déclenchés par l'utilisateur pour
  - Faire appel aux services métiers (BU)
  - Modifier l'état de l'interface utilisateur
  - Aider la progression de l'utilisateur dans sa tâche
- Filtrer les actions disponibles en fonction du contexte
- Effectuer des transformation simples (ex: l'utilisateur choisit un produit via son nom mais on transmet l'ID)
- Mettre des éléments en cache
- Gérer les aspects multi-langues (localization)
- Templating / theming

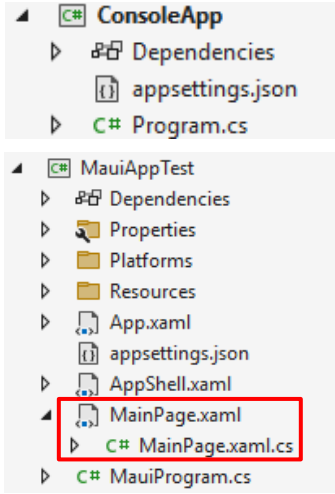
4

4

Organisation en couches

La couche présentation (3/4)





Pages en 2 parties:

1. **Apparence:** décrite en **XAML**
2. **Comportement**: implémenté en C# dans le **code-behind**

5

5

Organisation en couches

La couche présentation (4/4)

Couches de présentation complexes:

Dans des interfaces complexes ou les manipulations de l'utilisateur suivent un processus prévisible (ex: processus de commande en ligne) on peut mettre en œuvre des **UI Process Components** qui se chargent d'orchestrer l'exécution des processus.

6

6

Organisation en couches

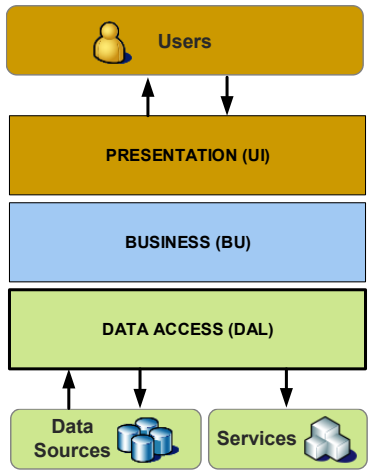
La couche d'accès aux données (1/7)

La **couche d'accès aux données** encapsule les spécificités des accès en lecture/écriture des différentes sources de données sous-jacentes (y compris externes).

On peut y trouver différents types de composants:

- **Modèle ORM**, comme l'Entity Framework
- **Classes customs**: Data Access Logic Components, repositories, …
- …

Evolue vers couche **infrastructure** (+ générique)



7

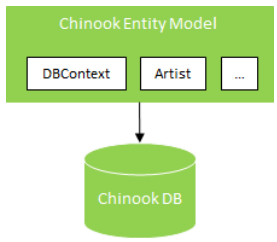
7

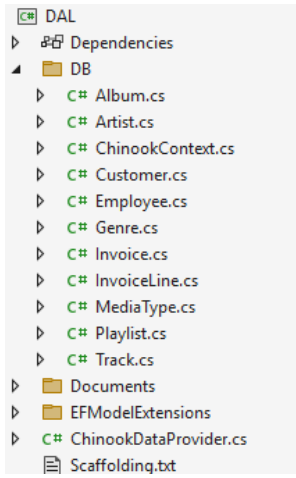
Organisation en couches

La couche d'accès aux données (2/7)

Entity Framework (ORM): Entity Data Model

- C'est l'Entity Data Model qui joue le rôle de couche d'accès aux données
- Il est composé de:
  - Classes d'entités associées aux tables
  - Classe DbContext
- On peut étendre les classes d'entité





8

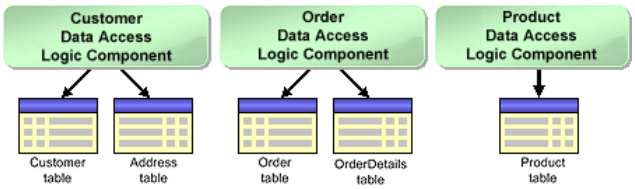
8

Organisation en couches

La couche d'accès aux données (3/7)

Classes DLAC (Data Access Logic Components):

- Encapsulent l'accès à une ou plusieurs tables d'1! datasource



- Fournissent des méthodes pour les opérations CRUD:
  - **Create** records in the database
  - **Read** records in the database (return business entity)
  - **Update** records in the database, by using revised business entity data supplied by the caller
  - **Delete** records in the database

9

9

Organisation en couches

La couche d'accès aux données (4/7)

Classes DLAC (Data Access Logic Components) (suite):

- Jouent d'autres rôles complémentaires
  - Fournir des méthodes qui implémentent de la logique métier liée à la BD (ex: récupérer le produit le plus vendu du mois)
  - Mapping d'arguments (ex: lien avec procédures stockées)
  - Gérer des transactions qui impliquent plusieurs MAJ dans une source de données unique (sinon c'est le rôle de la couche BU!)

10

10

Organisation en couches

La couche d'accès aux données (5/7)

Exemple de DLAC :

```
public class OrderDataProvider
{
    static string connString = ConfigurationManager.ConnectionStrings["NorthwindConnString"].ConnectionString;

    public static DataRow GetOrder(int orderID)...

    /// <summary>
    /// Retourne les lignes (détails) de la commande
    /// </summary>
    /// <param name="orderID"></param>
    /// <returns></returns>
    public static DataTable GetOrderDetails(int orderID)...

    /// <summary>
    /// Retourne le total des lignes de la commande (prix unitaire * quantité)
    /// </summary>
    /// <param name="orderID"></param>
    /// <returns></returns>
    public static decimal GetOrderTotal(int orderID)...
}
```

11

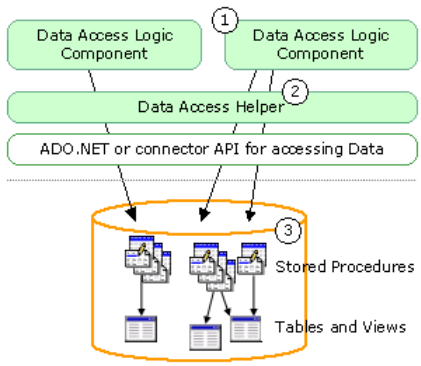
11

Organisation en couches

La couche d'accès aux données (6/7)

Couches Data complexes:

- Si beaucoup de DALC on peut mettre en place un composant d'aide à l'accès aux données (Data Access Helper) ou des classes utilitaires



12

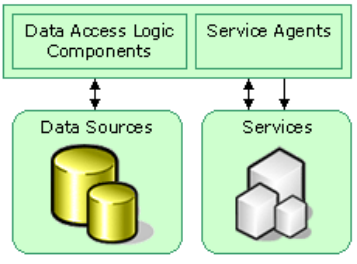
12

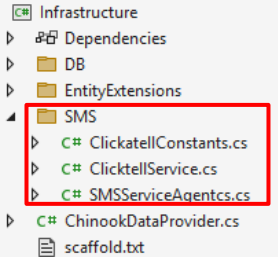
Organisation en couches

La couche d'accès aux données (7/7)

Couches Data complexes (suite):

- Si l'application doit utiliser des service ou API externes (ex: pour la validation d'un paiement ou l'accès à un CRM) on ajoute des **Services Agents** (agents de services) qui possèdent la logique nécessaire pour communiquer avec les services ou API.





13

13

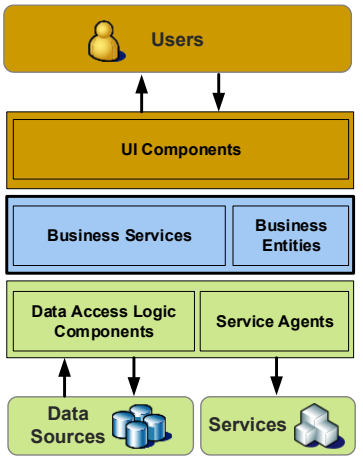
Organisation en couches

La couche métier (1/5)

La couche **business** (ou métier) contient la logique applicative ou logique métier de l'application (comment agir sur les données en fonction des actions des utilisateurs).

Elle est constituée des **Business Services** (classes métiers) chargées de réaliser les différentes tâches de l'application, par ex:

- Renvoyer une commandes client avec ses détails.
- Gérer la clôture d'une vente en ligne.
- Initier le processus de commande de tous les articles en rupture de stock.



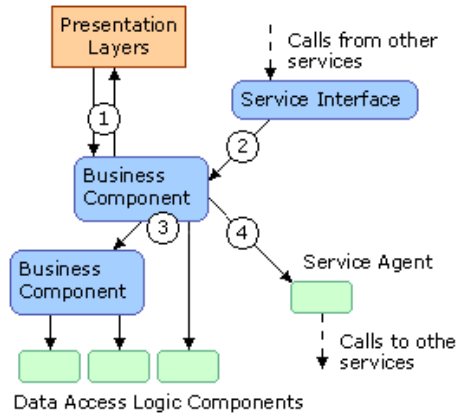
14

14

Organisation en couches

La couche métier (2/5)

Interactions dans la couche business



15

15

Organisation en couches

La couche métier (3/5)

Les Business Entities:

Les composants métiers manipulent des structures de données (simples ou complexes), indépendantes de la source de données et qui sont la représentations d'entités métier du monde réel (client, commande, ...): les **Business Entities**.

Elles peuvent revêtir plusieurs formes:

- Objets customs (= POCO: plain old CLR objects)
  - Business Entity Components
  - Business Entity Components with CRUD behaviors
- Entités de l'Entity Framework via
- XML
- JSON (JavaScript Object Notation) , ...

16

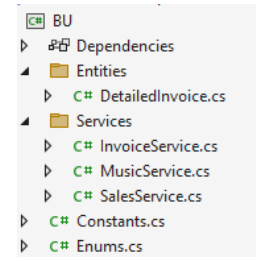
16

Organisation en couches

La couche métier (4/5)

Les **Business Entities** sont :

- Construites par la couche business sur base des données fournies par la couche d'accès aux données
- Créées par la couche DAL et propagées par la couche business



Certaines Business Entities sont transmises à la couche interface, par exemple, pour y être affichées et éditées par l'utilisateur. Après modification, l'entité business est renvoyée à la business layer qui décide de ce qu'il faut faire des modifications (par exemple demander à la DAL de MAJ la BD).

17

17

Organisation en couches

La couche métier (5/5)

Couches Business complexes:

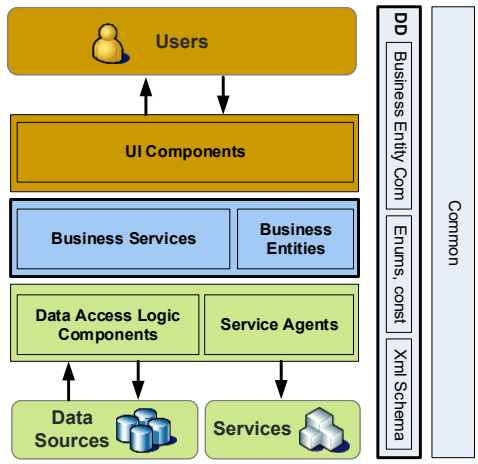
- Quand les processus métiers sont très complexes ils doivent parfois être orchestrés. On utilise alors des **Business Workflow Components**.
- Si des fonctionnalités métiers doivent être exposées en tant que service ou si la couche métier se situe sur un serveur indépendant, il faut fournir un point d'entrées via les **Service Interfaces** (interfaces de service).

18

18

Organisation en couches

Synthèse: architecture générale

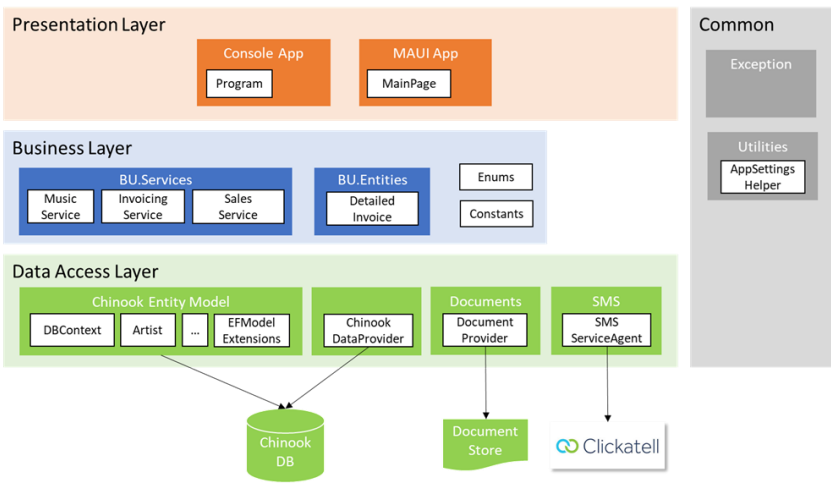


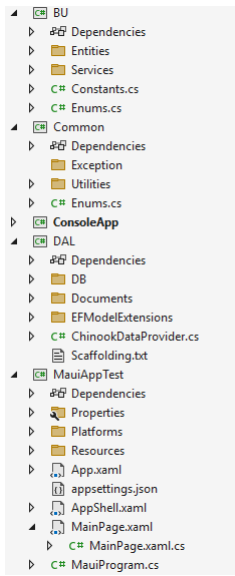
19

19

Organisation en couches

Architecture Workshop finale





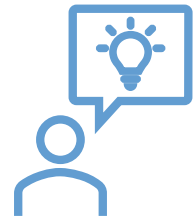
20

20

Exercices

Architectures applicatives

- Devoir – Architecture du projet : « Création des bases de la solution .NET et du schéma d'architecture du projet »
- Date de remise: 11/02/2026
- Révision lors de la séance de suivi du 16/02/2026 (en distanciel)



21